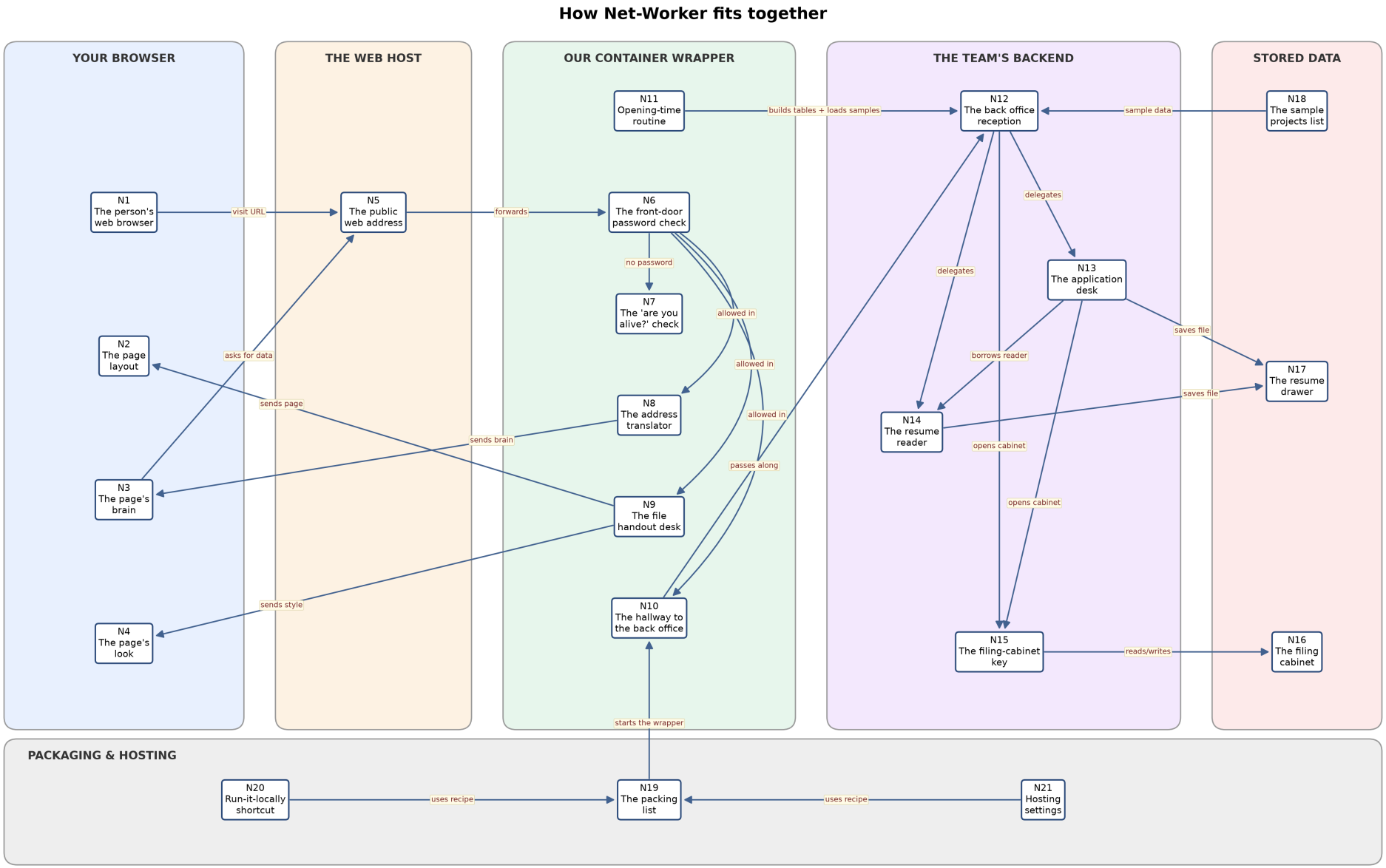
Net-Worker (ProjectHub)

02 - Plain-English Walkthrough

This is the companion to 01_Technical_Architecture_and_Syntax.pdf. Every section number and every box label (N1-N21) matches that document, so you can read them side by side.

In one sentence: ProjectHub is a small website where project leads post open roles and engineers find matches and apply; our container puts it online behind a team password without changing the team's code.

1. System graph



1. System graph - what each box means

ID	Box	What it does
N1	The person's web browser	Where everything starts and ends: someone types the Render address and sees the ProjectHub screens.
N2	The page layout	The skeleton of the screen: five tabs (Browse, Profile, Recommended, My Applications, Manager) and two pop-ups.
N3	The page's brain	Runs inside the browser. It reacts to clicks, asks the server for data, and redraws the screen with the answer.
N4	The page's look	Colours, spacing and layout. It changes nothing about data; it only makes the screens look right.
N5	The public web address	Render owns the public URL and the security certificate. It forwards each request into our container.
N6	The front-door password check	Every request must carry the shared team password, or the visitor gets a login prompt instead of the app.
N7	The 'are you alive?' check	The one door with no password, so Render can ping it to confirm the app is up.
N8	The address translator	Hands the browser app.js with the laptop-only address swapped for '/api', so the page talks to this same site.
N9	The file handout desk	Gives the browser the page files (HTML, CSS) exactly as they are in the repo.
N10	The hallway to the back office	Anything whose address starts with /api is passed through, unchanged, to the team's original backend.
N11	Opening-time routine	When the container starts it creates any missing tables and loads 10 fake projects if the database is empty.
N12	The back office reception	The team's main server file: lists projects, imports CSV files, records applications, updates statuses.
N13	The application desk	Handles 'apply with a resume' and 'show me my applications' requests.
N14	The resume reader	Checks that an uploaded resume is a real PDF/Word file and pulls the text out of it.
N15	The filing-cabinet key	Opens the database and knows how to build the four tables on first run.
N16	The filing cabinet	One file holding all projects, people and applications. On Render free tier it is wiped on every restart.
N17	The resume drawer	Uploaded resume files, renamed to random IDs so names can't clash or be guessed.
N18	The sample projects list	Ten clearly fake projects so a fresh site is never empty.
N19	The packing list	Recipe that packs only the code (never the real database or resumes) into one container image.
N20	Run-it-locally shortcut	Lets any teammate start the same container on a laptop with one command.
N21	Hosting settings	Written-down Render settings for later; today the service was set up by hand.

Why the arrows go this way

- Everything goes through **one web address**. The browser sends the password once and then reuses it, and there are no cross-site permission problems, because the page and its data come from the same place.
- The **password check sits before everything else** (except the health check), so nothing - not even the page layout - leaks to someone without the password.
- The team's backend is **plugged in, not edited**. The wrapper hands it requests under /api and redirects where it saves files, so teammates' code stays exactly as they wrote it.
- Data only **flows down** to the filing cabinet through one door (database.py), so there's a single place that decides where the data lives.

2. Request lifecycle: Render URL -> container -> backend

- **Step 1.** The browser finds Render and opens a secure, encrypted line.

- **Step 2.** Render passes the request into our container, where the web server picks it up.
- **Step 3.** The front door asks for the team password. Wrong or missing: the browser shows a login box.
- **Step 4.** Once typed, the browser remembers the password for this site, so later clicks just work.
- **Step 5.** The page and its 'brain' are handed out; the brain is told to call this same site for data.
- **Step 6.** As soon as the page opens, the brain asks the back office for the list of open projects.
- **Step 7.** The hallway passes the request to the team's backend, which knows what 'projects' means.
- **Step 8.** The backend opens the filing cabinet and pulls out every project still marked Open.
- **Step 9.** The answer travels back the same way it came, as a simple data list.
- **Step 10.** The brain keeps the list in memory and draws a card per project; searching filters that list without asking the server again.
- **Step 11.** Applying sends the form and resume together. The server checks it, saves the file, records the application, and undoes everything if any step fails.

3. File-by-file breakdown

backend/api/application_routes.py

The 'apply with resume' desk. It checks the person, project and file, saves the resume, records the application, and lists a person's past applications.

backend/api/main.py

The team's main server file. It sets up the backend, teaches it to read messy CSV column names, and answers the requests for projects and applications.

backend/api/resume_routes.py

The resume reader. It knows how to pull plain text out of PDF and Word files and enforces the 5 MB, PDF/DOCX-only rule.

backend/database/database.py

The filing-cabinet key: where the database file lives, how to open it, and the blueprint for its four drawers (tables).

deploy/Dockerfile

The packing list for the container. It deliberately copies code file by file so private data can't sneak in, and runs as a non-admin user.

deploy/demo_projects.csv

Ten fake projects used to fill an empty database.

deploy/docker-compose.yml

A one-line way to run the container on a laptop with a local test password.

deploy/render.yaml

Render's settings written down (Docker, free plan, health check, password asked at setup). Not wired up yet.

deploy/requirements.txt

The list of Python add-ons the container installs, with exact versions so every build is identical.

deploy/server.py

Our wrapper. It puts the page and the backend on one web address, adds the team password, moves saved data to a safe folder, and loads sample projects.

frontend/app.js - too long, see section 7

Everything the page does when you click: switching tabs, loading projects, scoring matches, sending applications, and the manager's CSV upload.

frontend/index.html

The page structure: the header, the five tabs, and the two pop-up windows.

frontend/styles.css

The visual design only. Safe to ignore when tracing data.

4. Endpoint catalogue

Address (on the live site)	What it's for	Used by page?
POST /api/applications/submit-with-resume	Apply to a project, with an optional resume file.	yes
GET /api/applications/my-applications	Shows everything one email address has applied to.	yes
GET /api/	Says hello - proves the backend is running.	no
GET /api/health	Backend's own 'I'm fine' check.	no
GET /api/projects	Lists every project still open.	yes
GET /api/projects/{project_id}	Shows one project in full.	yes
POST /api/projects/upload-csv	Manager uploads a spreadsheet of new projects.	yes
POST /api/applications	Older way to apply (no resume file). Not used by the page.	no
GET /api/applications/{application_id}	Looks up one application by number. Not used by the page.	no
GET /api/projects/{project_id}/applications	Lists who applied to a project. Not used by the page yet.	no
PATCH /api/applications/{application_id}/status	Manager changes an application's status. Not used by the page yet.	no
POST /api/resumes/upload	Uploads a resume on its own. Not used by the page.	no
GET /healthz	Render's alive check - the only route without a password.	infra
GET /app.js	Hands out the page's brain with the address fixed up.	infra

5. Data models

- **projects** - one row per open role: title, client, skills needed, level, location, start date, status.
- **employees** - one row per person who has applied, found by email. Holds their skills and resume text.
- **applications** - links a person to a project, with a status (Submitted, Under Review, Accepted, Rejected). The same person can apply to the same project only once.
- **matches** - meant to store match scores, but nothing writes to it yet; scores are worked out in the browser.
- **Request forms** - the backend checks incoming forms against a fixed shape (name, valid email, etc.) and rejects anything that doesn't fit before it touches the database.

6. State management

- The page remembers your skills, the project list and the project you picked only until you refresh.
- Match percentages are worked out on your own computer; the server's 'matches' table sits empty.
- The server forgets you between requests; everything worth keeping goes into the database.
- Saved data lives in one database file and a resume folder. On the free plan both are wiped whenever the site restarts.
- There are no personal logins yet. Anyone with the team password can look up applications by typing an email.

7. Files over 800 lines

frontend/app.js is longer than our 800-line limit. It works, but it does five jobs in one file (talking to the server, remembering state, and drawing four different tabs), which makes it hard for several people to edit at once. The

plan is to split it into one small file per job.

8. Risks & observations

- There are two ways to apply in the code; the page uses only one. Keeping both doubles the upkeep.
- The cabinet key is loaded twice under two names, so the wrapper has to update both copies.
- The backend says 'any website may call me', which is loose if it's ever run without the wrapper.
- The backend uses an old start-up hook; that's why our wrapper does the start-up job itself.
- Picking a resume on the Profile tab doesn't send it anywhere, and the separate upload endpoint saves orphan files.
- One address only works because of the order lines appear in; moving them would break it.
- Private data is still in the shared code history; it never reaches the live site, but it should be removed (Round 2).

Explaining the system in 30 seconds

A visitor opens the Render link and is asked for the team password. Our container then hands them the page. The page asks the team's backend, through the /api hallway, for open projects, which come out of a single database file. Applying sends the form and resume back the same way; the backend checks it, files it, and replies. On the free plan, all saved data resets to ten sample projects whenever the site restarts.